

CM3 - C Language

Representations, programs, and builds

Louis Ledoux

2026-07-03

Table of contents

1	Course Shape	2
1.1	The rule for this deck	2
1.2	Today as a C course	2
1.3	What this version demonstrates	2
1.4	Sources and adaptation	2
2	Values And Bits	2
2.1	Representation is a contract	2
2.2	Fixed-width integers	3
2.3	Unsigned arithmetic wraps	3
2.4	Signed overflow is different	4
3	First Program	4
3.1	Anatomy of main	4
3.2	argv is just data from the shell	4
3.3	printf: integer first	5
3.4	printf: then floating point	5
3.5	Format strings are contracts	5
3.6	Common printf specifiers	5
4	Compilation	5
4.1	C is compiled for a target machine	5
4.2	The build pipeline	6
4.3	Quarto can execute a C build at render time	6
4.4	Static trace is still useful	6
5	Multi-File Projects	6
5.1	A header is the public contract	6
5.2	Include declarations, not duplicate definitions	7
5.3	Longer example with scrolling	7
6	Build Automation	8
6.1	Separate compilation	8
6.2	A Makefile records dependencies	8
6.3	Full Makefile with tests	8
6.4	The tab matters	8
6.5	What make changes	8
7	Undefined Behavior	9
7.1	Quiz: uninitialized local variable	9
7.2	Undefined behavior breaks reasoning	9
7.3	Use braces	9
7.4	C truthiness	9
8	Tools	10
8.1	Warnings are part of the course	10

8.2	Scrollable diagnostics trace	10
8.3	Debugger workflow	10
8.4	Memory checker workflow	10
9	Export Strategy	10
9.1	One source, three outputs	10
9.2	Print handouts from RevealJS	11
9.3	What to keep in the final course	11

1 Course Shape

1.1 The rule for this deck

Horizontal movement continues the lecture path.

Vertical movement opens details, examples, or exercises on the current topic.

In printable formats, the same vertical detail slides become normal pages.

1.2 Today as a C course

- C programs are data plus representation choices.
- Source files are translated, linked, and executed.
- Tools matter: compiler warnings, make, debugger, memory checker.
- The deck itself is a testbed for teaching code well.

1.3 What this version demonstrates

Feature	RevealJS	Beamer PDF	Typst handout
Vertical detail slides	interactive	printed as pages	printed as sections
Progressive line high-lights	stepped	static fallback	static fallback
Code annotations	hover/selectable	below code	below code
Scrollable long code	live only	avoid for final	avoid for final
Executed C example	executed at render	executed at render	executed at render

1.4 Sources and adaptation

This version combines:

- your imported CM3 material on files, compilation, make, tools, and style;
- the Berkeley CS61C lecture you added, especially number representations, C vs Java, printf, and undefined behavior;
- Quarto/RevealJS features that are useful for teaching code.

The Berkeley material is used as inspiration for topic order and examples, not as a slide import.

2 Values And Bits

2.1 Representation is a contract

A bit pattern has no meaning by itself.

0b11111111 can mean:

- unsigned integer: 255
- signed 8-bit integer: -1
- character byte: implementation dependent
- raw mask: all bits set

The program works only if writer and reader agree on the representation.

```

#include <stdint.h>
#include <stdio.h>

int main(void) {
    uint8_t byte = 0xff;
    printf("%u\n", byte);
}

```

2.2 Fixed-width integers

When the size matters, say it.

```

1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5      uint8_t byte = 0xff;
6      int32_t temp = -12;
7
8      printf("byte=%u temp=%d\n", byte, temp);
9      return 0;
10 }

```

The point is not to ban int. The point is to make students ask whether size is part of the contract.

2.3 Unsigned arithmetic wraps

```

1  #include <stdint.h>
2  #include <stdio.h>
3
4  int main(void) {
5      uint8_t x = 255;
6      x = x + 1;
7      printf("%u\n", x);
8  }

```

The result is 0 because unsigned arithmetic is modulo 2^N .

Quick check

What does this program print?

```

#include <stdint.h>
#include <stdio.h>

int main(void) {
    uint8_t x = 250;
    x = x + 10;
    printf("%u\n", x);
}

```

i Answer

It prints 4.

With an 8-bit unsigned integer, arithmetic is modulo 256, so $250 + 10 = 260$, and $260 \% 256 = 4$.

2.4 Signed overflow is different

```
#include <limits.h>
#include <stdio.h>

int main(void) {
    int x = INT_MAX;
    printf("%d\n", x + 1);
}
```

This is not the same story as unsigned wraparound. Signed overflow is undefined behavior in C.

⚠ Warning

For teaching: this is a good moment to separate “what my machine did today” from “what C promises”.

3 First Program

3.1 Anatomy of main

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("hello, CM3\n");
    return 0;
}
```

Line 1

Declaration of the standard I/O interface.

Line 3

Entry point; argc and argv describe command-line arguments.

Line 4

Formatted output.

Line 5

Exit status returned to the shell.

3.2 argv is just data from the shell

```
$ ./show_args 1 hi
argv[0] = ./show_args
argv[1] = 1
argv[2] = hi
```

```
1  #include <stdio.h>
2
```

```

3  int main(int argc, char *argv[]) {
4      for (int i = 0; i < argc; i++) {
5          printf("argv[%d] = %s\n", i, argv[i]);
6      }
7      return 0;
8  }

```

3.3 printf: integer first

```

int n = 42;
printf("value = %d\n", n);

```

%d asks printf to read the next argument as a signed decimal integer.

3.4 printf: then floating point

```

double x = 42.0;
printf("value = %f\n", x);

```

%f asks printf to read the next argument as a floating-point value.

3.5 Format strings are contracts

```

1  #include <stdio.h>
2
3  int main(void) {
4      double x = 42.0;
5      printf("as int: %d\n", x);
6      printf("as double: %f\n", x);
7  }

```

The compiler may warn, but printf itself trusts the format string.

3.6 Common printf specifiers

Specifier	Meaning	Typical argument
%d	signed decimal integer	int
%u	unsigned decimal integer	unsigned int
%x, %X	hexadecimal integer	unsigned int
%f	floating point	double
%s	string	char *
%p	address	void *
%%	literal percent sign	none

4 Compilation

4.1 C is compiled for a target machine

C:

- source is translated before execution;
- output depends on architecture and system ABI;
- compiler warnings are part of the feedback loop.

Java/Python comparison:

- Java source becomes bytecode, then JIT/runtime work happens;
- Python usually compiles to bytecode at runtime;
- portability is paid for by a runtime.

4.2 The build pipeline

```
1  $ gcc -E hello.c -o hello.i
2  $ gcc -S hello.i -o hello.s
3  $ gcc -c hello.s -o hello.o
4  $ gcc hello.o -o hello
```

- Preprocess: expand directives like `#include`.
- Compile: turn C into assembly.
- Assemble: turn assembly into an object file.
- Link: combine objects and libraries into an executable.

4.3 Quarto can execute a C build at render time

The generated RevealJS/Beamer/Typst output is static, but this cell compiles and runs C while the document is rendered.

```
$ gcc -Wall -Wextra hello.c -o hello
$ ./hello
hello from compiled C
```

4.4 Static trace is still useful

Use a plain bash code block when the goal is a readable transcript rather than live execution.

```
$ gcc -Wall -Wextra hello.c -o hello
$ ./hello
hello from compiled C
```

This is deterministic, portable, and easy to print.

5 Multi-File Projects

5.1 A header is the public contract

```
#ifndef COUNTER_H
#define COUNTER_H

extern int counter;
int next_value(void);

#endif

#include "counter.h"

int counter = 0;

int next_value(void) {
    return ++counter;
}
```

5.2 Include declarations, not duplicate definitions

```
1  #include "counter.h"
2  #include <stdio.h>
3
4  int main(void) {
5      printf("%d\n", next_value());
6      return 0;
7  }
```

The header lets each .c file compile independently while agreeing on shared declarations.

5.3 Longer example with scrolling

In printable exports, prefer splitting this example into several shorter slides or keeping it as an appendix.

```
#ifndef COUNTER_H
#define COUNTER_H

extern int counter;
int next_value(void);

#endif

#include "counter.h"

int counter = 0;

int next_value(void) {
    return ++counter;
}

#include "counter.h"
#include <stdio.h>

void report_value(const char *label) {
    printf("%s: %d\n", label, counter);
}

#include "counter.h"

void report_value(const char *label);

int main(void) {
    report_value("initial");
    next_value();
    report_value("after one step");
    next_value();
    report_value("after two steps");
    return 0;
}
```

6 Build Automation

6.1 Separate compilation

```
1  $ gcc -Wall -Wextra -c main.c -o main.o
2  $ gcc -Wall -Wextra -c counter.c -o counter.o
3  $ gcc -Wall -Wextra -c report.c -o report.o
4  $ gcc main.o counter.o report.o -o demo
5  $ ./demo
```

Object files let us rebuild only what changed.

6.2 A Makefile records dependencies

```
1  CC = gcc
2  CFLAGS = -Wall -Wextra -g
3  OBJ = main.o counter.o report.o
4  EXEC = demo
5
6  all: $(EXEC)
7
8  $(EXEC): $(OBJ)
9  ____$(CC) $(OBJ) -o $@
10
11 main.o: main.c counter.h
12 ____$(CC) $(CFLAGS) -c $< -o $@
13
14 counter.o: counter.c counter.h
15 ____$(CC) $(CFLAGS) -c $< -o $@
16
17 report.o: report.c counter.h
18 ____$(CC) $(CFLAGS) -c $< -o $@
```

6.3 Full Makefile with tests

For printable exports, keep the long Makefile in a handout or repository file. The slide deck should show the dependency pattern, not every target.

6.4 The tab matters

```
clean:
____rm -f *.o demo
```

The command line under a target starts with a real tab character.



Tip

For class, make the invisible tab visible once, then stop talking about it and let make enforce the rule.

6.5 What make changes

```
$ make
gcc -Wall -Wextra -g -c main.c -o main.o
gcc -Wall -Wextra -g -c counter.c -o counter.o
gcc -Wall -Wextra -g -c report.c -o report.o
```

```
gcc main.o counter.o report.o -o demo

$ touch report.c
$ make
gcc -Wall -Wextra -g -c report.c -o report.o
gcc main.o counter.o report.o -o demo
```

Only the affected object and the final executable are rebuilt.

7 Undefined Behavior

7.1 Quiz: uninitialized local variable

```
1  #include <stdio.h>
2
3  int main(void) {
4      int x;
5      printf("%d\n", x);
6      return 0;
7  }
```

What should students answer?

- Not “0”.
- Not “a random number” as a language guarantee.
- The correct C answer is: the program has undefined behavior.

7.2 Undefined behavior breaks reasoning

Undefined behavior can:

- appear stable on one machine;
- change after optimization;
- change when a debugger is attached;
- disappear when a print is added;
- survive tests and fail later.

The teaching rule: compile with warnings, remove undefined behavior first, then debug logic.

7.3 Use braces

```
1  if (ready)
2      start();
3
4  if (ready) {
5      start();
6  }
7  cleanup();
```

The second form makes scope visible and prevents future edits from changing control flow by accident.

7.4 C truthiness

```
if (0) {
    puts("false");
}
```

```

if (42) {
    puts("true");
}

if ("hello") {
    puts("also true");
}

```

In C17, include `<stdbool.h>` if you want `bool`, `true`, and `false`.

8 Tools

8.1 Warnings are part of the course

```

$ gcc -Wall -Wextra -Wpedantic -g ub.c -o ub
ub.c: In function 'main':
ub.c:5:5: warning: 'x' is used uninitialized [-Wuninitialized]

```

Do not treat warnings as decoration. In this course, students should read them from the top.

8.2 Scrollable diagnostics trace

Printable exports should use a shorter diagnostic excerpt and discuss the full trace in the handout.

8.3 Debugger workflow

```

$ gcc -Wall -Wextra -g main.c counter.c report.c -o demo
$ gdb ./demo
(gdb) break main
(gdb) run
(gdb) next
(gdb) print counter
(gdb) backtrace

```

Good debugger slides are usually command traces plus a small code fragment, not screenshots.

8.4 Memory checker workflow

```

$ valgrind --tool=memcheck ./demo
==12345== HEAP SUMMARY:
==12345==    in use at exit: 16 bytes in 1 blocks
==12345== total heap usage: 1 allocs, 0 frees
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 16 bytes in 1 blocks

```

The tool is useful after the program compiles and the failure is runtime behavior.

9 Export Strategy

9.1 One source, three outputs

```

$ PATH="$PWD/venv/bin:$PATH" quarto render cm3.qmd --to revealjs
$ PATH="$PWD/venv/bin:$PATH" quarto render cm3.qmd --to beamer
$ PATH="$PWD/venv/bin:$PATH" quarto render cm3.qmd --to typst

```

Output	File	Use
RevealJS	cm3.html	live lecture with navigation, fragments, scroll, animation
Beamer	cm3-beamer.pdf	slide PDF close to the classroom deck
Typst	cm3-handout.pdf	printable reading handout from the same source

9.2 Print handouts from RevealJS

RevealJS can be printed from the browser using `?print-pdf`.

```
$ chromium --headless --no-sandbox \
  --print-to-pdf=cm3-reveal-print.pdf \
  "file://$PWD/cm3.html?print-pdf"
```

```
$ pdfjam cm3-reveal-print.pdf --nup 1x2 --paper a4paper --outfile cm3-2up.pdf
$ pdfjam cm3-reveal-print.pdf --nup 2x2 --paper a4paper --outfile cm3-4up.pdf
$ pdfjam cm3-reveal-print.pdf --nup 3x2 --landscape --paper a4paper --outfile cm3-6up.pdf
```

These PDFs are for students who print. They should not depend on scrolling.

9.3 What to keep in the final course

Keep:

- code filenames and line highlights;
- vertical detail slides for optional explanations;
- one executed example per topic when it proves something;
- short command traces for terminal stories;
- Typst handouts for printable reading.

Avoid:

- long scrollable slides as the only source of information;
- terminal styling that competes with syntax highlighting;
- animations that carry essential meaning with no PDF fallback.